

Choosing the number of clusters with the elbow method

WILDA Team 4 · ACS Work Integrated Learning, Data Analytics · Stage 2 Clustering Analysis · Ramesh Kunwar

1. Purpose

K-Means splits customers into a fixed number of groups. It places k centres in the data, puts every customer with the nearest centre, moves each centre to the average of the customers it collected, and repeats until the centres stop moving. The result is k groups whose members sit closer to their own centre than to any other.

K-Means cannot work out k on its own. The number of clusters is an input, fixed before training starts, and the algorithm returns exactly that many groups whatever number it is given. Ask for two and it returns two. Ask for ten and it returns ten. Neither answer says whether that count suits the data. This document sets out how we chose $k = 4$ for the 5,634 customers in the training set, and what the trained model produced.

2. Data used

The model is fitted on the 5,634 rows of `Data_Preparation/train_set.csv`, using these 10 features: gender, SeniorCitizen, Dependents, tenure, PhoneService, MultipleLines, InternetService, MonthlyCharges, Contract_One_year, Contract_Two_year. The data preparation stage already scaled **tenure** and **MonthlyCharges** with a StandardScaler fitted on this same training set, so no rescaling happens here.

- **Churn is excluded.** Stage 3 predicts churn. A cluster built from the churn column would carry the answer inside the feature, and any model trained on it would score well for the wrong reason. The pipeline asserts that the target is absent and that exactly 10 features remain. Churn is used only after clustering, to measure the churn rate of each finished segment.
- **Only the training set is used to fit.** The 1,409 test rows are assigned afterwards with `model.predict`, which puts each one with its nearest existing centre without moving that centre. This keeps the test set unseen, matching how the scaler was fitted in the preparation stage.

3. The elbow method

Inertia is the within-cluster sum of squares. It adds up the squared distance from every customer to the centre of its own cluster. A small inertia means customers sit close to their centres.

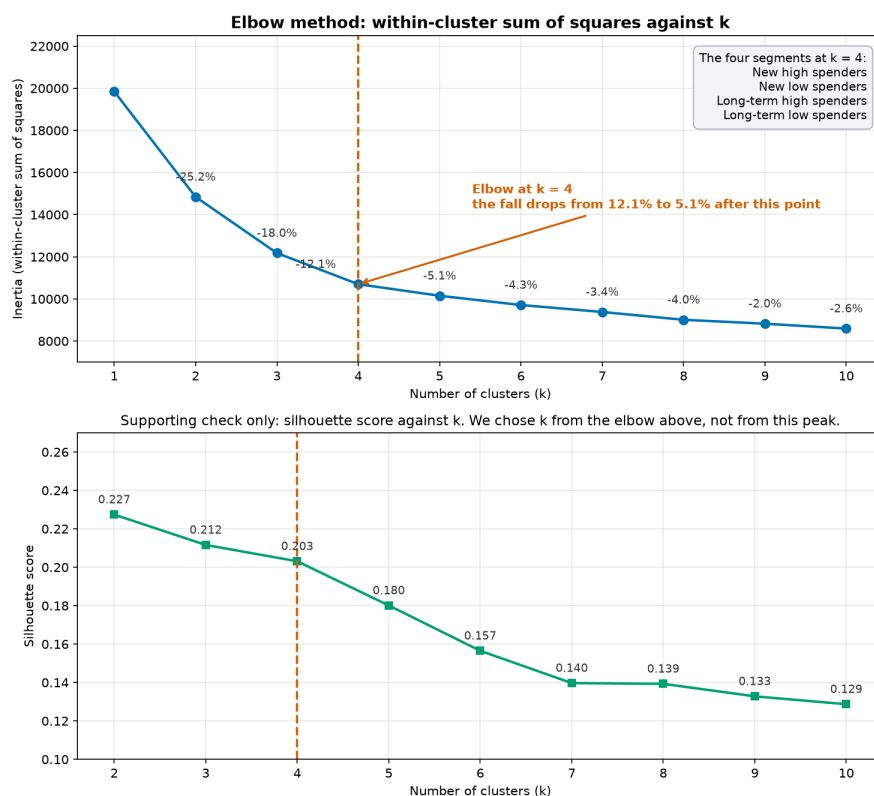
Inertia always falls as k rises, which is why its value alone cannot choose k . Adding a centre gives every customer one more centre to be near, so no customer can end up further away. At k equal to the number of customers, every customer is its own cluster and inertia reaches 0. Picking the k with the smallest inertia would therefore always pick the largest k tested, and each cluster would describe fewer customers than the one before.

The elbow method reads the size of each fall instead of the value. Early clusters split groups that really are separate, so inertia drops sharply. Once the real groups have been found, another cluster only cuts an existing group in half, and the drop becomes small. The elbow is the last k before the drops go flat. We report the percentage fall at each step, so the reading is a number rather than a judgement about the shape of a line.

4. Results

The pipeline fits K-Means for every k from 1 to 10 with `n_init=10` and `random_state=42`. The silhouette score is computed on all 5,634 training rows, with no sampling. It is not defined at $k = 1$, because a single cluster has nothing to be compared against.

k	Inertia	Fall in inertia	Silhouette
1	19,845.66		
2	14,844.21	25.20%	0.2274
3	12,178.36	17.96%	0.2116
4	10,700.86	12.13%	0.2031
5	10,152.73	5.12%	0.1801
6	9,712.96	4.33%	0.1565
7	9,382.54	3.40%	0.1397
8	9,011.45	3.96%	0.1393
9	8,826.27	2.05%	0.1328
10	8,594.34	2.63%	0.1287



Inertia against k, with the elbow at k = 4 marked and each point labelled with its percentage fall. The silhouette score below is a supporting check.

5. Choice of k

We chose k = 4. Inertia falls 25.2% at k = 2, 18.0% at k = 3 and 12.1% at k = 4. The next step gives only 5.1%, and every step after that gives less than 4.3%. The fall from 12.1% to 5.1% is the sharpest change in the sequence, so k = 4 is the last cluster that buys a large improvement.

The silhouette score is highest at k = 2, at 0.2274, and falls to 0.2031 at k = 4. We did not follow that peak, for two reasons. The assignment asks for the elbow method, and silhouette is recorded here as a check rather than as the rule. Two clusters would also split the base into little more than newer and longer standing customers, which gives the business one group to contact and one to leave alone.

At k = 4 the segments separate on churn, from 4.8% to 48.7% against 26.54% overall. New high spenders hold 1,775 customers and 864 of the 1,495 churners. Long-term low spenders hold 966 customers and 46 churners. That is a difference the business can act on, because it says where to spend a retention budget and where not to.

6. The trained model

The final model is a KMeans with `n_clusters=4`, `n_init=10` and `random_state=42`. `n_init=10` runs the algorithm 10 times from different starting centres and keeps the run with the lowest inertia, because a single run can settle on a poor set of centres. `random_state` fixes the starting points, so every run of the pipeline returns the same model. It reaches an inertia of 10,700.86 and a silhouette score of 0.2031.

Segment	Customers	% of base	Churners	Churn rate	Tenure (months)	Charge (\$)
New high spenders	1,775	31.5%	864	48.7%	14.9	81.10
New low spenders	1,381	24.5%	342	24.8%	10.5	32.10
Long-term high spenders	1,512	26.8%	243	16.1%	58.7	93.80
Long-term low spenders	966	17.1%	46	4.8%	54.5	34.80

The training step fits the model on the training features, then assigns both sets:

```
model = KMeans(n_clusters=OPTIMAL_K, n_init=N_INIT, random_state=RANDOM_STATE)
model.fit(X_train)

train_labels = model.labels_          # the training rows the model saw
test_labels = model.predict(X_test)   # unseen rows, nearest centre
```

The fitted model is saved with joblib, so later stages load it instead of training their own:

```
joblib.dump(model, OUT_DIR / "kmeans_model.pkl")
```

`kmeans_model.pkl` stores the four centres, the settings above and the 10 feature names in `feature_names_in_`, which does not include Churn.

7. Limitations

The four segments differ on tenure and monthly charge, and barely differ on anything else. Fibre optic runs only from 41.6% to 45.4% across the four segments. The share on a one or two year contract runs from 44.3% to 46.4%. Phone service is the only 0/1 feature that separates the segments at all.

This follows from how K-Means measures distance. tenure and MonthlyCharges are z-scores, so they spread roughly from -2 to 3. The other eight columns hold 0 or 1 and can differ by at most 1. The two wider columns therefore account for most of the distance between any two customers, and the model groups on them. The segments should be read as tenure and spending groups, not as statements about contract type or internet type.

Stage 3 should keep contract type and internet type as predictors in their own right. This clustering does not capture them, so the segment label cannot stand in for them. The segment in `cluster_assignments.csv` is one extra feature for that model, alongside the original columns rather than instead of them.

The silhouette score of 0.2031 is worth stating plainly. A score near 0 means neighbouring clusters sit close together rather than in separated groups, which the tenure and charge chart also shows. The segments sort customers into useful action lists. They are not evidence that four distinct customer types exist in this data.